Simulating Logistics Systems

Lesson 2/15

João Flávio de Freitas Almeida

LEPOINT: Laboratório de Estudos em Planejamento de Operações Integradas
Departmento de Engenharia de Produção

Universidade Federal de Minas Gerais
Escola de Engenharia

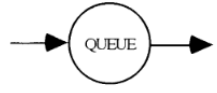
UFMG, Belo Horizonte, Brasil
20.08.2025

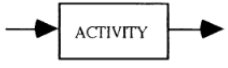
Outline of this lecture

Outline of this lecture

Activity Cycle Diagram (ACD) [Paul, 1993]

- This modeling tool is useful for teaching the main elements of a simulation model, however, limited for large-scale real-world problems.





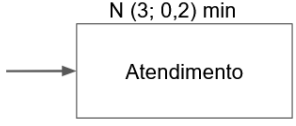
ENTITY	STATES	SYMBOL
Man	Drink	
	Wait	
Barmaid	Pour	
	Idle	
Glass	Drink from	
	Empty	
	Pour into	
	Full	

Entities [Paul, 1993]

- Elements in the system whose behavior we wish to observe.
- Entities are not represented individually, but by their classes.
- The DCA should show the path that each entity class travels within the system.
- These paths are represented by distinct lines (arrows) for each class of entities.
- Entities can be either permanent or temporary.
- Permanent entities remain in the system at all times, have a defined quantity, and can be considered resources that enable certain tasks.
- Temporary entities pass through the system. Their quantity depends on the form of entry, and they are usually the most important.

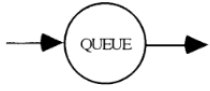
Activities [Paul, 1993]

- Represent the activities of entities, i.e., the tasks they perform.
- They consume time, which is predetermined.
- Duration of each activity: Can be deterministic or random;
- At least one entity must be present for execution.
- Graphical representation: rectangle with activity name in the center.
- Graphical representation: the expression placed just above the activity box.



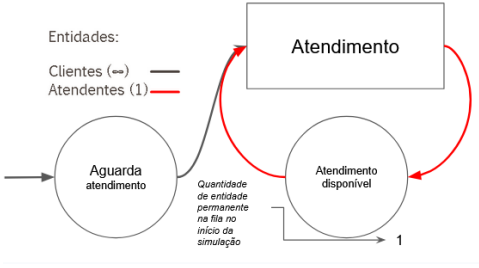
Queues [Paul, 1993]

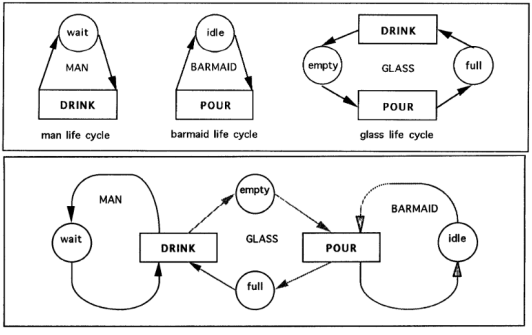
- They represent entities in a passive state, i.e., when they wait for a condition to occur before continuing on their path.
- They consume time, but the amount of time is not predetermined.
- In DCA, each queue is exclusive to a single type of entity.
- The graphical representation is a circle with the name of the queue in the center.



Representation [Paul, 1993]

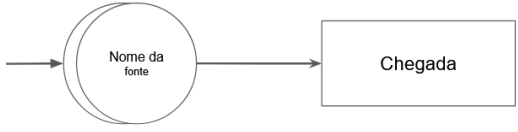
- Entities + Activities + Queues.
- Entities pass through activities and queues alternately. Life cycle of entity class.
- *Life cycles of three entities (man, waiter, glass) in the system (bar) and their DCA.*





Sources and Sinks [Paul, 1993]

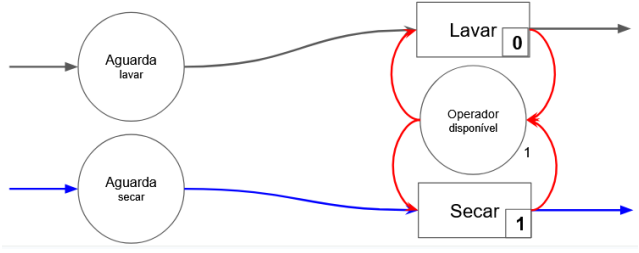
- Temporary entities must enter the system, undergo certain activities, and exit the system. The entry and exit of these entities are represented by sources and sinks;
- The graphical representation is made by two overlapping circles, and the source can also be the sink;
- A source can be seen as an infinite capacity queue that always has entities waiting to enter the system;
- After a source, there must always be an arrival activity.



Additional elements of ACD [Paul, 1993]

Priority in executing activities:

- It is always necessary to set execution priorities when two or more activities compete for the same entity;
- The highest priority has a value of zero, followed by the lowest priorities (1, 2, 3);
- Graphical representation: the priority should be placed in a rectangle inside the activity box, in the lower right corner.
- Note: *when the operator is available and has items to wash and dry, they prioritize washing.*



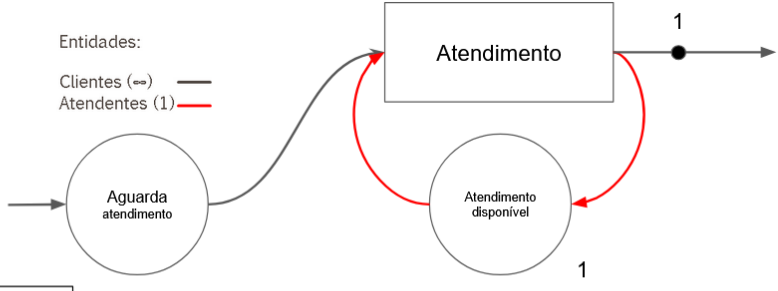
Additional elements of ACD [Paul, 1993]

Attributes and **Variables**:

- **Attributes**: represent characteristics of entities.
  - They must be numerical;
  - Each **entity** has its own value;
- **Variables**: represent characteristics of the system.
  - They must be numerical;
  - They belong to the **system**.
- Graphical representation:
  - Points in ACD where values change (assignment points). Should be indicated.
  - These points must always be after some activity and before the next queue;
  - The points must be identified by numbers and the assignment rule must be explained in a table next to the ACD.

Additional elements of ACD [Paul, 1993]

Attributes and **Variables**:



Atribuições:

1. Atributo N = N/2

   Variável Z = Z + N

Obs: *Em um mesmo ponto de atribuição podem ser alterados valores de mais de um atributo e/ou variável*

Additional elements of ACD [Paul, 1993]

Deviations (multiple paths)

- When an entity can take more than one path, the rules must be clear.
- Types of deviation: Probabilistic or Conditional;
- They must occur after an *activity* and before the next *queue*.
- The possible paths must be identified by letters and their **rules** must be made explicit in a **table** next to the ACD.

Desvios:

A ou B?
A ← 30%
B ← 70%

Probabilístico

Desvios:

A ou B?
Se altura > 1,70m, Então: A
Senão: B

Condicional

Additional elements of ACD [Paul, 1993]

Queue discipline

- FIFO – First In, First Out (default). That is, first come, first served.
- LIFO – Last In, First Out;
- Priority – Reads an attribute (Priority = higher or lower attribute value);
- Random – No priority;
- *Graphical representation: indicated next to the queue (FIFO is the default and does not need to be specified)*

Mandatory requirements for a DCA [Paul, 1993]

1. Alternation between **Activities** and **Queues** in the process of each entity class;
2. The duration of each activity must be indicated next to each activity;
3. **Queues** are exclusive to a single entity class;
4. **Queues** *without explicit disciplines* are considered **FIFO**;
5. **Deviations** must have their rules specified in a table attached to the ACD;
6. **Assignment points** explicit specified in a table attached to the ACD;
7. **Temporary entity** must begin and end at a Source/Sink;
8. **Permanent entities** must have their quantities specified next to their queue.
9. **Activities** that simultaneously **compete for resources** (permanent entities) must have their priorities specified.

Implicit logic in ACD [Paul, 1993]

1. The *sine qua non* condition for starting an activity is the existence of at least one entity in each of the queues preceding the activity;
2. The start of an activity is characterized by the removal of one, and only one, entity from each queue preceding the activity;
3. Entities are retained during the execution time of the activity, which is sampled in advance;
4. The end of an activity is characterized by the release of the retained entities and the placement of each of them in their respective queues, subsequent to the activity;
5. For these reasons, it is necessary to have queues before and after each activity;
6. It is also necessary that each queue be exclusive to only one type of entity.

Outline of this lecture

The process method

It is a way of modeling the system as entities (customers, patients, parts) pass through the system. In the process method, customers "live" their journey in the system, waiting and using resources. The simulation advances in time, modeling each entity's journey.

- Rather than controlling the global list of events, as in the event-driven method, or controlling activities, and in the 3 Phases method, the model defines each entity's life cycle.
  - What it **does** (*activity*);
  - Where it **waits** (*queue*);
  - What resources it **uses** (*resource*);
  - For how long.
- This DES approach is used in the SimPy (Python) library.

The process method

Typical logical flow:

1. **Create** entities (e.g., hospital patients).
2. **Describe** the process for each entity:
   - Arrive
   - Wait for resource
   - Use resource
   - Leave
3. **Resources** have implicit queues.
4. *The simulation engine advances in time by jumping to the next relevant event (end of service, arrival, etc.).*

The process method

A numerical example:

- Situation: A patient triage by a nurse.
- Arrivals: every 5 minutes (deterministic).
- Service time: 8 minutes (deterministic).
- Simulate 3 patients.

1. Arrive at the nurse
2. If the nurse is busy $\rightarrow$ wait in line
3. When the nurse is free $\rightarrow$ begin service
4. When service is complete $\rightarrow$ leave

The process method: Step-by-step simulation

Time (min)	Event	Queue	Server	Observation
0	Patient 1 arrives, starts service	0	Busy	Service ends at $t = 8$
5	Patient 2 arrives, joins the queue	1	Busy	Waits until $t = 8$
8	Patient 1 leaves, Patient 2 starts	0	Busy	Service ends at $t = 16$
10	Patient 3 arrives, joins the queue	1	Busy	Waits until $t = 16$
16	Patient 2 leaves, Patient 3 starts	0	Busy	Service ends at $t = 24$
24	Patient 3 leaves	0	Free	End of simulation

- **Total time in the system:**

$$\begin{aligned} &\text{Patient 1: } 8 \text{ min} \quad (0 \to 8) \\ &\text{Patient 2: } 11 \text{ min} \quad (3 \text{ waiting} + 8 \text{ service}) \\ &\text{Patient 3: } 14 \text{ min} \quad (6 \text{ waiting} + 8 \text{ service}) \end{aligned}$$

- **Average time in the system:** $\frac{8+11+14}{3} = 11$ min
- **Average waiting time in queue:** $\frac{0+3+6}{3} = 3$ min

Outline of this lecture

ACD: Example 1 [Pinto, 2016]

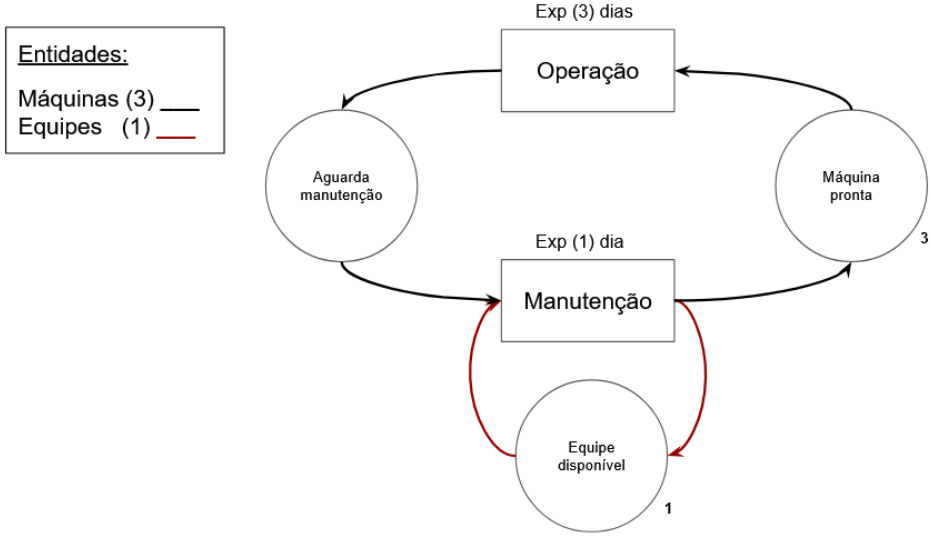
Machines operating continuously.

- A company operates three machines in a specific sector of its industrial plant. The machines operate continuously, interrupting their operation only for corrective maintenance. The time between failures is described by an exponential distribution with an average of 3 days. Maintenance is performed by a single team and its duration follows an exponential distribution with a mean of 1 day.
- We want to simulate this problem **to** evaluate the time that the machines are idle waiting for maintenance and to estimate the average occupancy of the maintenance team. To do this, build the conceptual model of the system using activity cycle diagrams.

Dynamic presentation (Pt-Br) in Google Sheets

ACD: Example 1 [Pinto, 2016]

Machines operating continuously. Dynamic presentation (Pt-Br) in Google Sheets



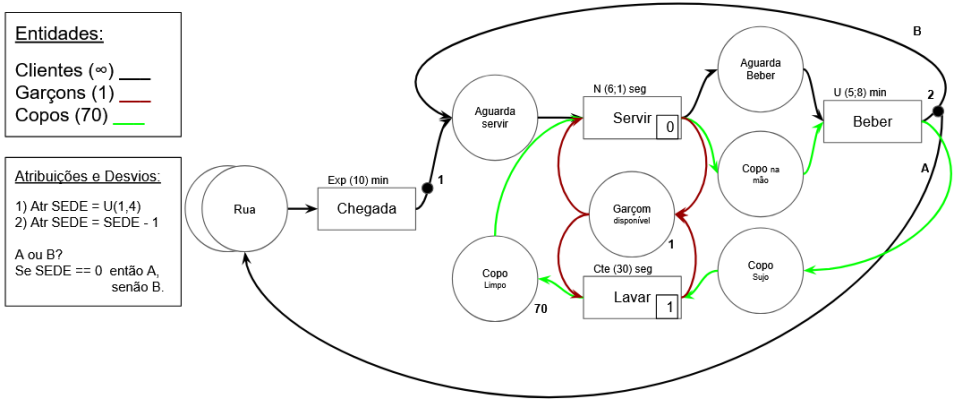
ACD: Example 2 [Pinto, 2016]

Bar problem. Dynamic presentation (Pt-Br) in Google Sheets

- In a bar, customers arrive from the street to drink beer, in quantities that vary randomly depending on how thirsty they are. The intervals between arrivals are exponentially distributed with an average of 10 minutes. The number of glasses each customer drinks is defined upon arrival, through the THIRST attribute. A customer's THIRST varies according to a discrete uniform distribution with a minimum of 1 and a maximum of 4 glasses. Upon arriving at the bar, a customer will wait their turn to be served. Once served, the activity, whose duration follows a normal distribution with an average of 6 seconds and a standard deviation of 1 second, the customer will drink their glass. The time to drink a glass is uniformly distributed with values between 5 and 8 minutes. This cycle will repeat until the customer's thirst is quenched. A waiter is responsible for serving customers and washing used glasses. Serving customers also requires that a clean glass be available. Washing glasses takes a time that can be considered constant and equal to 30 seconds. It is also assumed that the bar has 70 glasses.
- Develop a model using the ACD to represent the system.

ACD: Example 2 [Pinto, 2016]

Bar problem. Dynamic presentation (Pt-Br) in Google Sheets



ACD: Example 3 [Pinto, 2016]

Telephone central. Dynamic presentation (Pt-Br) in Google Sheets

- A telephone central receives calls at random intervals according to an exponential distribution with an average interval between calls of 4 seconds. The average duration of a conversation is 120 seconds, also following an exponential distribution. The exchange has a **limited capacity** corresponding to the number of available trunks, which is equal to **30**. A call that encounters a congested system (all trunks busy) is lost.
- That said, we are ask to construct the ACD for this system, as we want to build a simulation program **to** estimate the average number of busy trunks and the percentage of lost calls.
- Furthermore, knowing that the percentage of lost calls should be limited to 5%, we want to know how many trunks would be needed to meet this level of service and what the average occupancy of the central would be in this case.

ACD: Example 3 [Pinto, 2016]

Telephone central. Dynamic presentation (Pt-Br) in Google Sheets

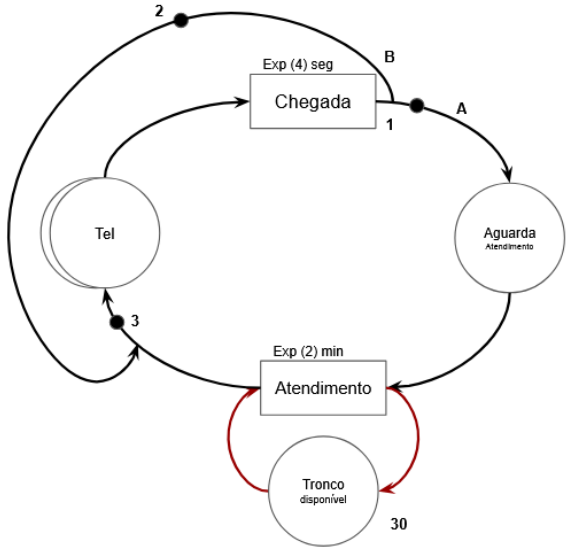
Entidades:

Chamadas (∞) __
Troncos (30) ___

Atribuições e Desvios:

1) Var Cheg = Cheg + 1
2) Var Perd = Perd + 1
3) Var PecPerd = 100*Perd/Cheg

A ou B?
Se TamFila(Tronco Disp) > 0 então A,
senão B.



ACD: Example 3.1 [Pinto, 2016]

Telephone central. Dynamic presentation (Pt-Br) in Google Sheets

- Exercise 3.1: Suppose that a call may be lost when the system is congested (all trunks are busy), which occurs with a 30% probability, or else be reconnected (return) within 10 seconds. There is no pre-established limit to the number of returns a call may have.

ACD: Example 3.1 [Pinto, 2016]

Telephone central (3.1). Dynamic presentation (Pt-Br) in Google Sheets

Entidades:

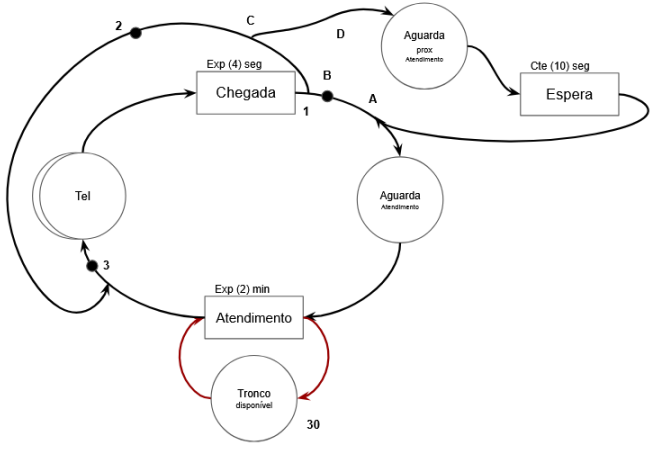
Chamadas (∞)
Troncos (30)

Atribuições e Desvios:

1) Var Cheg = Cheg + 1
2) Var Perd = Perd + 1
3) Var PecPerd = 100*Perd/Cheg

A ou B?
Se TamFila(Tronco Disp) > 0 então A,
senão B.

C ou D?
C ← 30%
D ← 70%



ACD: Example 3.2 [Pinto, 2016]

Telephone central. Dynamic presentation (Pt-Br) in Google Sheets

- Exercise 3.2: Suppose that a call, encountering a congested system (all trunks busy), is directed to an auxiliary exchange with another 10 trunks of capacity. If the auxiliary exchange is also congested, the call will be lost (there is no return in this case).

ACD: Example 3.2 [Pinto, 2016]

Telephone central (3.2). Dynamic presentation (Pt-Br) in Google Sheets

Entidades:

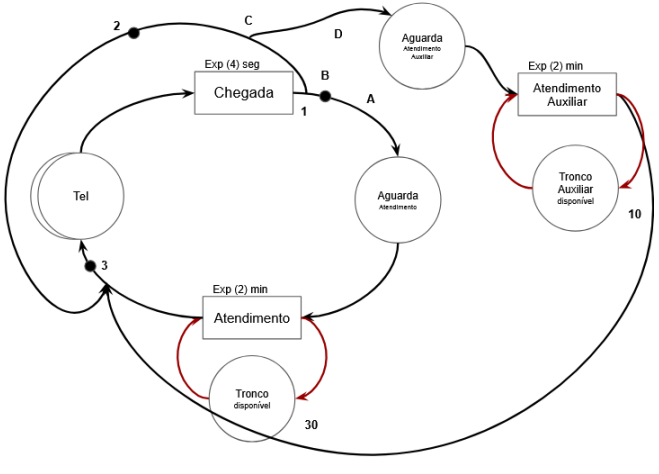
Chamadas (∞) __
Troncos (30) ____

Atribuições e Desvios:

1) Var Cheg = Cheg + 1
2) Var Perd = Perd + 1
3) Var PecPerd = 100*Perd/Cheg

A ou B?
Se TamFila(Tronco Disp) > 0 então A,
senão B.

C ou D?
Se TamFila(Tronco Aux Disp) > 0 então C,
senão D.



Outline of this lecture

Python Practice

- Download (or clone) *the course Github repository*;
- Create a virtual environment: `py -m venv venv`
- Activate the virtual environment: `. /venv/Scripts/activate.ps1` (**venv**)
- Install requirements: `pip install -r requirements.txt`
- Install ipython: `pip install ipython`
- Open `00-Suplementary-Material-Python-Course`
- Run Ipython: `> ipython`
- Open a file to study: e.g. `01-Sintaxe.py`
- Select each line (or more) and `SHIF + ENTER`

References

Paul, R. J. (1993).
Activity cycle diagrams and the three-phase method.
In *Proceedings of the 25th conference on Winter simulation*, pages 123–131.

Pinto, L. R. (2016).
Simulação por eventos discretos.
Notas de Aula., 1.